

La MMU cognitive

Le projet CCOS¹

2026-06-26

Abstract

La mémoire de travail d'un agent de code à base de LLM est gérée de façon opaque : une pile de récupération (RAG, GraphRAG, frameworks d'agents) pagine un dépôt dans une fenêtre bornée et mute cette fenêtre de manière probabiliste au fil d'une session ; aussi, lorsqu'un agent dérive après des dizaines d'étapes, il est impossible de dire *pourquoi* ni *quand* sa représentation du projet s'est corrompue. CCOS traite la mémoire de l'agent comme un système d'exploitation traite la RAM : le code source est analysé en un graphe causal *event-sourced*, les nœuds sont scorés et paginés sous un budget de jetons, et chaque transition d'état est enregistrée dans un journal chaîné par hachage qui se rejoue bit pour bit. Nous donnons une définition précise et déterministe d'une *région causale* (une composante connexe du graphe des liens causaux inter-fichiers) et de la distance causale (un plus court chemin pondéré), et prouvons que les régions et la fenêtre paginée sont une fonction pure du graphe, se reconstruisant exactement au rejeu — la propriété qui rend la mémoire d'un agent *auditable* et *déboguable par voyage dans le temps*. Nous demandons ensuite si cette structure causale fait de CCOS un *meilleur récupérateur* qu'une référence de base, et rapportons — honnêtement — que non. Mesurée sur 70 commits réels de correction de bogues issus de trois bibliothèques (« crates ») matures, la sélection causale **fait jeu égal avec un simple récupérateur lexical TF-IDF** et lui est inférieure à budget serré ; sur des bogues réels, les fichiers d'un correctif partagent leur vocabulaire, si bien que la similarité lexicale les trouve aussi, et une preuve de concept amorçant CCOS depuis une trace de crash est elle aussi battue par un RAG-sur-le-message-d'erreur (les messages d'erreur de Rust nomment la cause). Une simulation sans LLM *sépare* bien les deux lorsqu'une cause est construite pour être lexicalement dissemblable de son symptôme, et un premier essai embarqué avec un modèle de 7 milliards de paramètres le reproduit — mais ce régime ne se produit pas sur du vrai code. La contribution n'est donc *pas* une meilleure récupération. C'est l'infrastructure qui manque structurellement à une pile de récupération : une mémoire d'agent *déterministe, rejouable, auditable* dans laquelle on peut rembobiner l'état exact du contexte à n'importe quelle étape et le rejouer sous d'autres paramètres (*débugage par voyage dans le temps*). À notre connaissance, CCOS est le *premier* système à traiter la *mémoire de travail elle-même* d'un agent comme un sous-système transactionnel — déterministe, chaîné par hachage, rejouable bit-à-bit et *déboguable a posteriori* : un enregistreur de vol de l'attention, avec un point d'arrêt sur l'étape exacte où la bonne information a été évincée de la fenêtre. Nous publions le moteur, le harnais de validation honnête et cet article, en étiquetant partout ce qui est *prouvé* (structure formelle, déterminisme, rejeu), *mesuré* (couverture de récupération — où CCOS ne bat *pas* le RAG lexical sur des bogues réels), et le seul axe où CCOS *gagne* : **l'efficacité**. Exécuté de bout en bout sur de vraies corrections mono-fichier avec un modèle de 30B et une nouvelle tentative avec le compilateur dans la boucle, CCOS et le RAG résolvent à *égalité* (2/10), mais CCOS le fait sur 776 jetons de contexte contre 5366 pour le récupérateur lexical — une réduction de $6,9 \times$ (et de $4,1$ à $9,1 \times$ sur 51 correctifs issus de trois crates, sans modèle), car la région causale s'arrête au working set au lieu de remplir un top- k jusqu'au budget. CCOS n'est donc pas un meilleur récupérateur ou solveur, mais un récupérateur plus *frugal* et *auditable* — il s'auto-calibre, sans k à régler. (Sur sa propre arborescence source, la sélection par région couvre le voisinage à k sauts d'une tâche avec un rappel de 0,97 contre 0,35 pour la sélection plate, avec $\approx 48\%$ de jetons en moins ; les régions sont connectées en interne à 95,5%.)

Introduction

Un agent de code à base de LLM opérant sur un dépôt doit décider de façon répétée *quoi mettre dans la fenêtre de contexte*. Le motif dominant est la récupération : encoder des fragments de code, les classer par rapport à la tâche, et concaténer le top- k jusqu'à épuisement d'un budget de jetons . Cela traite le contexte comme une liste classée unidimensionnelle. Deux modes de défaillance s'ensuivent. D'abord, la *dilution* : à mesure que les tâches s'allongent, la fenêtre accumule du code globalement saillant mais sans rapport avec la tâche, et les modèles prêtent mal attention au milieu des longs contextes . Ensuite, l'*incohérence* : les k fragments les mieux classés ne forment pas nécessairement une unité connexe de raisonnement — une fonction peut être paginée sans ses appelants, ses sites d'erreur ou les données dont elle dépend.

Les systèmes d'exploitation ont affronté le problème analogue il y a des décennies et ont répondu par le *working set* : l'ensemble des pages récemment référencées qui doivent rester résidentes . CCOS adopte l'analogie pour le contexte LLM : le code source est analysé en un graphe causal, les nœuds sont scorés, et une « fenêtre de contexte » bornée est paginée comme la RAM ↔ VRAM. Contrairement à une pile de récupération, chaque transition est enregistrée dans un journal d'événements chaîné par hachage, si bien que la mémoire n'est pas une boîte noire mais un artefact *auditable et rejouable*. Nos contributions sont :

1. Un **modèle formel et déterministe de régions de contexte** (§4, §5) : la distance causale comme plus court chemin pondéré, l'appartenance à une région comme composante connexe du graphe des liens causaux inter-fichiers, et un théorème de déterminisme — les régions et la fenêtre paginée sont une fonction pure du graphe, donc une session se reconstruit bit pour bit depuis son journal d'événements chaîné par hachage (rejeu à l'épreuve de la falsification).
2. Des **sessions d'agent event-sourced avec débogage par voyage dans le temps** (§6) : chaque opération cognitive (ingestion, signal d'échec, rappel) est journalisée ; l'état exact du contexte à toute étape est restructurable ; et un rappel peut être *rejoué sous d'autres paramètres* pour demander si l'agent aurait mieux décidé — la capacité qui manque structurellement à une pile de récupération probabiliste. À notre connaissance, c'est le premier traitement de l'*assemblage du contexte lui-même* comme un sous-système rejouable et déboguable a posteriori — un *enregistreur de vol de l'attention* — doté d'un *point d'arrêt d'éviction* qui nomme l'étape et l'opération exactes où la vraie cause a été expulsée de la fenêtre budgétée.
3. Un **harnais de validation honnête et un résultat négatif** (§10) : sur 70 commits réels de correction de bogues, la sélection causale *ne bat pas* un récupérateur lexical TF-IDF pour placer les fichiers d'un correctif dans la fenêtre (elle fait jeu égal, et perd à budget serré), et un pivot par trace de crash est battu par un RAG-sur-le-message-d'erreur. Nous le rapportons clairement — cela déplace la valeur de CCOS de la *récupération* vers l'*auditabilité*.
4. Une **mesure de localité sans LLM** (§8) avec des chiffres réels et reproductibles, et un protocole falsifiable de condition *suffisante* (Phase 4) que nous spécifions mais laissons ouvert.

Travaux connexes

Génération augmentée par récupération.

Le RAG augmente un modèle paramétrique d'un magasin non paramétrique et récupère des passages par requête . Self-RAG ajoute des jetons de réflexion qui contrôlent la récupération et critiquent les générations . Ces approches opèrent sur des fragments *indépendants* ; la cohérence entre les éléments récupérés n'est pas modélisée.

Récupération sensible aux graphes et à la structure.

GraphRAG construit un graphe de connaissances d'entités et répond aux requêtes globales en résumant la structure communautaire . Pour le code, les graphes de propriétés unifient AST, flot de contrôle et flot de données . Les régions de CCOS s'inscrivent dans cette lignée mais visent la *pagination* : quelle sous-structure connexe rendre résidente pour une tâche, sous un budget de jetons, avec une éviction déterministe.

Mémoire d'agent.

MemGPT présente le LLM comme un OS qui pagine entre une fenêtre en contexte et un stockage externe ; les Generative Agents récupèrent des souvenirs scorés par récence, importance et pertinence — les mêmes facteurs que CCOS agrège en une température de région. LangGraph structure les agents en graphes d'étapes à état ; il orchestre le flot de *contrôle*, là où CCOS structure la *mémoire*. Les deux sont complémentaires.

Gestion de la fenêtre de contexte.

L'éviction du cache KV maintient résidents les jetons « gros frappeurs » ou puits , et PagedAttention applique la pagination OS au cache KV . Ces techniques agissent au niveau du jeton à l'intérieur d'une passe avant ; CCOS agit au niveau *sémantique* à travers une session d'agent.

Le substrat de contexte causal

CCOS analyse le code source en un *graphe de mémoire causale* dirigé $G = (V, E)$. Un nœud $v \in V$ est un fichier, un module, un symbole ou une dépendance externe, avec des champs scalaires $\text{imp}(v)$ (importance de base), $\text{fail}(v) \in [0, 1]$ (pertinence d'échec) et $\text{rec}(v) \in [0, 1]$ (récence). Une arête $e = (u \rightarrow w) \in E$ porte un poids $w(e) \in (0, 1]$ et un type (contenance, dépendance, référence, causation). Le noyau attribue à chaque nœud un score causal

$$\text{score}(v) = \text{clamp}(0.15 \text{ imp}(v) + 0.50 \text{ fail}(v) + 0.30 \text{ rec}(v) + 0.05 \ln(1 + \text{acc}(v)), 0, 1),$$
où $\text{acc}(v)$ est le compteur d'accès. Les fautes se propagent le long des arêtes, fail décroît avec une horloge logique, et chaque transition d'état est ajoutée à un journal d'événements chaîné par hachage permettant un replay déterministe . Les identifiants de nœuds sont à espace de noms (`file:p`, `mod:p:n`, `use:p:path`, `sym:p:n`, `dep:root`) ; le fichier propriétaire d'un nœud est récupérable depuis son identifiant. CCOS v0.2 pagine les nœuds de plus haut score : une politique plate et 1-D. Nous rendons désormais la sélection spatiale.

Régions de contexte

Distance causale

Définition 1 (Distance causale). Soit $\hat{G}$ le multigraphe non orienté sur V induit par E , et attribuons à chaque arête le coût $c(e) = -\ln w(e) \geq 0$, de sorte qu'un lien causal plus fort soit un pas plus court. La distance causale $d_c(u, v)$ est le coût total minimum sur tous les chemins $u-v$ dans $\hat{G}$, et $+\infty$ s'il n'en existe aucun. La distance en sauts non pondérée $\text{hops}(u, v)$ est définie de façon analogue avec des coûts unitaires.

$c(e) \geq 0$ puisque $w(e) \leq 1$, donc d_c est une véritable métrique de plus court chemin sur chaque composante connexe (non négative, symétrique, inégalité triangulaire). Le voisinage causal à k sauts d'une cible t est

$$\mathcal{N}_k(t) = \{v \in V : \text{hops}(t, v) \leq k\}.$$

$\mathcal{N}_k(t)$ est la vérité-terrain « ce qui est causalement pertinent pour une tâche en t » utilisée dans l'évaluation (§8).

Appartenance à une région

Les concentrateurs de dépendances externes (p. ex. `dep:std`) relient presque tout et ne doivent pas effondrer le graphe en une seule composante. Nous les séparons donc.

Définition 2 (Lien causal inter-fichiers). Pour un nœud non externe v , soit $\phi(v)$ son fichier propriétaire. Deux fichiers f, g sont directement liés, noté $f \approx g$, si et seulement s'il existe une arête $(u \rightarrow w) \in E$ avec $\phi(u) = f$, $\phi(w) = g$, $f \neq g$, et ni u ni w n'est un nœud de dépendance externe.

Définition 3 (Région). Soit $\approx^*$ la clôture réflexive-transitive de $\approx$ sur l'ensemble des clés de fichiers. Une région est l'ensemble de tous les nœuds dont les fichiers appartiennent à une même classe d'équivalence de $\approx^*$; tous les nœuds de dépendance externe forment une région supplémentaire. Deux nœuds appartiennent à la même région si et seulement si leurs fichiers sont dans la même composante connexe du graphe des liens causaux inter-fichiers.

Proposition 1 (Les régions partitionnent les nœuds). Les régions de la Définition 3 forment une partition de V : chaque nœud appartient à exactement une région.

Proof. $\approx^*$ est une relation d'équivalence sur les clés de fichiers, donc ses classes partitionnent les clés ; le seau externe est une classe distincte par construction. Associer chaque nœud non externe à la classe de son fichier et chaque nœud externe à la classe externe est une fonction totale vers des blocs disjoints, donc une partition. $\square$

Par défaut (uniquement des arêtes de contenance et d'importation), chaque fichier source est sa propre région. Une véritable dépendance inter-fichiers ou un échec propagé fusionne des fichiers en une seule région multi-fichiers — la « zone de connaissance » qu'un agent devrait réveiller ensemble.

Scalars de région

Pour une région R d'ensemble de membres M :

$$\text{heat}(v) = 0.5 \text{ score}(v) + 0.3 \text{ fail}(v) + 0.2 \text{ rec}(v),$$

$$\text{temp}(R) = \text{clamp}\left(\frac{1}{|M|} \sum_{v \in M} \text{heat}(v), 0, 1\right),$$

$$\text{dens}(R) = \frac{|\{e \in E: \text{les deux extrémités} \in M\}|}{|M|}.$$

La température mesure à quel point une région est « éveillée » ; la densité est sa cohésion causale interne (arêtes internes par membre). L'activation réchauffe une région (temp + = 0.25, plafonné) et enregistre un tic logique ; une étape de refroidissement multiplie les températures par un facteur de décroissance et évince les régions sous un plancher.

Politique d'admission dynamique

Le seuil statique 0.6 devient une fonction de la pression de jetons $u \in [0, 1]$ (la fraction utilisée du budget) et de la complexité de tâche $\kappa \in [0, 1]$:

$$\theta(u, \kappa) = \text{clamp}(0.6 + 0.3u - 0.2\kappa, 0.05, 0.95),$$

$$a(R) = 0.55 \text{ temp}(R) + 0.30 \frac{\text{dens}(R)}{1 + \text{dens}(R)} + 0.15 \kappa,$$

$$\text{admit}(R) \Leftrightarrow a(R) \geq \theta(u, \kappa).$$

Une région chaude et cohésive peut être admise même là où le 0.6 statique la rejetterait ; une fenêtre presque pleine élève θ pour que seules les régions les plus chaudes entrent.

Déterminisme et rejou

Théorème 1 (Déterminisme régional). *La partition en régions, chaque scalaire de région des équations [eq:temp]–[eq:dens], et l'historique d'activation sont des fonctions pures, indépendantes de l'ordre, du graphe G et de la suite d'événements d'activation (logiquement horodatés). Par conséquent, étant donné le journal d'événements d'une session, l'état du moteur se reconstruit à l'identique : si G' est le graphe reconstruit depuis le journal et L ses événements de région, alors $\text{replay_from}(G', L)$ égale le moteur en direct qui a produit L .*

Esquisse de preuve. Le clustering énumère les nœuds et arêtes dans un ordre trié et calcule les composantes connexes par un parcours en largeur trié, donc sa sortie est indépendante de l'ordre d'itération du hachage. Les équations [eq:score], [eq:temp] et [eq:dens] sont une arithmétique déterministe sur les champs des nœuds et un décompte d'arêtes qualifiantes. L'activation lit une horloge logique (un compteur), jamais l'heure murale, et l'événement RegionActivated émis enregistre le tic exact. La reconstruction re-clusterise G' (état de base identique, puisque $G' = G$ structurellement) et applique les activations enregistrées dans l'ordre du journal ; chaque étape est la même fonction pure des mêmes entrées, donc l'état résultant est identique. Le test d'intégration $\text{replay_reconstructs_identical_engine}$ vérifie $\text{engine} = \text{replay_from}(G', L)$ par égalité structurelle.  □ □

Cela étend les garanties existantes de CCOS : le journal d'événements principal est chaîné par hachage et à l'épreuve de la falsification, si bien que l'historique des régions est auditable, et un test de 10 000 cycles n'exhibe aucune dérive du nombre de régions ni

des températures.

Une frontière d'entrée auditable.

Le même substrat déterministe et rejouable durcit le texte qu'un agent ingère. Les vecteurs d'injection par caractères cachés — surcharges bidirectionnelles (l'attaque *Trojan Source*, CVE-2021-42574), formatage de largeur nulle, et le bloc Unicode *Tags* servant à la contrebande d'ASCII invisible — sont dé-obfusqués à l'ingestion en littéraux explicites et visibles ([U+202E RL0]) plutôt que silencieusement supprimés, et les *findings* sont consignés dans le même journal chaîné par hachage, de sorte qu'un rejeu reproduit l'état nettoyé. Un classifieur linéaire en log-espace en aval — la forme close du Naïve Bayes multinomial, $\text{logit} = b + W \cdot X$ sur un vecteur de caractéristiques par *hashing trick*, ses poids verrouillés dans un blob vérifié par somme de contrôle — ajoute un signal d'injection *déterministe et décomposable forensiquement* (un red-team en données réservées mesure $F_1 = 0,90$; précision 0,87, rappel 0,93). Nous l'affirmons délibérément : c'est un *signal, pas une solution* — un modèle sac-de-caractéristiques est contourné par paraphrase, et aucune passe au niveau caractère ne traite l'injection sémantique ; la vraie mitigation est la séparation des privilèges dans l'hôte. La contribution n'est pas un détecteur inédit, mais que la dé-obfuscation et le score héritent de la propriété distinctive de CCOS : toute décision de sécurité est reproductible et auditable jusqu'à la caractéristique exacte qui l'a fait basculer.

Sessions event-sourced et débogage par voyage dans le temps

Le déterminisme n'est pas seulement une propriété de correction ; il est le fondement de la capacité qui, comme notre évaluation l'argumentera, distingue CCOS d'une pile de récupération. Une *AgentSession* enregistre chaque opération cognitive qu'un agent effectue sur sa mémoire — *Ingest*, *SignalFailure*, *Recall* — comme une chronologie ordonnée. Parce que chaque opération est une fonction déterministe de l'état antérieur, la mémoire après les n premières opérations se reconstruit exactement en rejouant celles qui mutent (*replay_to(n)*) : on peut *rembobiner l'esprit de l'agent* à n'importe quelle étape.

L'opération qui compte pour le débogage est le contrefactuel. *recall_what_if(n, q, b)* rejoue la mémoire jusqu'à l'étape n et relance un rappel avec une requête q ou un budget de jetons b différents, renvoyant la fenêtre que l'agent *aurait* vue. Quand un agent émet un mauvais correctif à l'étape 15, un opérateur peut rejouer son contexte exact à l'étape 14, élargir le budget ou changer l'ancre, et observer si la décision s'améliore — un débogueur en boucle fermée et reproductible pour la mémoire de travail d'un agent.

Un point d'arrêt sur l'attention.

Le rejeu localise aussi la *dérive*. Le point d'observation *missing* parcourt la chronologie pour trouver l'étape exacte où un nœud donné — typiquement la vraie cause d'un échec — a été expulsé de la fenêtre budgétée par la pression concurrente, en nommant l'opération déclenchante et l'écart de jetons ; une vue *energy* complémentaire révèle la migration de chaleur causale à travers le graphe qu'un diff au niveau fichier manque. C'est, de fait, un *point d'arrêt sur l'attention d'un agent* : l'instant précis où la bonne

information a quitté la fenêtre. Une pile de récupération opaque ne peut montrer que le contexte final, corrompu ; CCOS peut désigner l'étape — et l'opération — où cela a déraillé.

Une pile RAG ou de framework mute son magasin de façon probabiliste au cours d'une session et ne conserve aucun journal de transitions canonique et rejouable, de sorte que la même question (*pourquoi, et quand, la représentation s'est-elle corrompue ?*) n'y a pas de réponse. Nous ne prétendons pas que cela améliore le succès des tâches ; nous prétendons que c'est une propriété que les alternatives n'ont pas, et que c'est le lieu honnête de la contribution de CCOS (§10).

Implémentation

Le moteur fait ≈ 600 lignes de Rust léger en dépendances, en couche au-dessus du graphe, sans modification du parseur, du garde, du constructeur incrémental ou du cœur event-sourcing. La surface publique est : `ContextRegionEngine` (`cluster_nodes`, `initialize_regions`, `activate_region`, `tick_cooldown`, `replay_from`), les types de données `ContextRegion` / `ContextPoint` / `ContextWindow`, la `ContextPolicy`, et cinq nouvelles variantes d'événement (`RegionCreated`, `RegionActivated`, `RegionMerged`, `RegionEvicted`, `ContextWindowGenerated`). Une CLI `ccos regions` expose le clustering, l'activation et le rapport de localité. L'ensemble de la crate se compile sans avertissement sous `clippy -D warnings` et passe 364 tests.

Évaluation : ce que nous pouvons mesurer aujourd'hui

Nous séparons les résultats *mesurés* (cette section, entièrement reproductible et sans LLM) des gains *hypothétiques* au niveau de l'agent (§10).

Dispositif.

Nous opérons sur la propre arborescence source de CCOS ($|V| = 705$ nœuds, $|E| = 822$ arêtes, donnant 26 régions). Pour un nœud cible t , nous prenons $\mathcal{N}_k(t)$ avec $k = 2$ comme vérité-terrain et comparons deux stratégies de sélection au budget de taille de la région : **plate** — les $|R|$ meilleurs nœuds par score (CCOS v0.2 / récupération classée classique) — et **région** — les membres de la région de t . Nous rapportons la précision causale $|\mathcal{S} \cap \mathcal{N}_k|/|\mathcal{S}|$, le rappel $|\mathcal{S} \cap \mathcal{N}_k|/|\mathcal{N}_k|$, et les jetons que chaque stratégie nécessite pour couvrir $\mathcal{N}_k$. Tous les chiffres sont émis par `scripts/region_benchmark.sh` et sont déterministes.

Localité sur `src/` ($k = 2, 12$ cibles). La sélection par région couvre 97% du voisinage causal d'une tâche contre 35% pour la sélection plate, avec $\approx 48\%$ de jetons en moins pour une couverture égale.

Stratégie	précision causale	rappel causal
plate (v0.2 / classée)	0.021	0.347

Stratégie	précision causale	rappel causal
région (v0.3)	0.057	0.972

Cohésion et coût.

Les régions atteignent une **densité causale moyenne de 0,955** (Éq. [eq:dens], normalisée) — elles sont presque entièrement connectées en interne, c.-à-d. de véritables grappes causales plutôt que des groupes de fichiers arbitraires. Construire la carte des régions pour 705 nœuds prend ≈ 20 ms (54,5 constructions/s) ; une seule activation fait ≈ 20 ms.

Lecture honnête.

La précision absolue est faible (0,06) parce que le parseur v0.2 n'émet que des arêtes de contenance et d'importation, donc $\mathcal{N}_k(t)$ est minuscule (souvent 2 à 3 nœuds) tandis qu'une région couvre un fichier entier. Les gains robustes et réels sont le *rappel* (0,97 contre 0,35) et l'*efficacité en jetons* ($\approx 48\%$) : une région contient de façon fiable le voisinage causal d'une tâche et paie moins de jetons pour le faire. Des arêtes sémantiques plus riches (graphe d'appels / flot de données, item P1.3 de la feuille de route) affinerait $\mathcal{N}_k$ et constituent le principal levier pour élever la précision ; nous le signalons plutôt que de le cacher.

Simulation d'hypothèse sous un oracle déclaré (sans LLM)

La thèse centrale — *la mémoire causale par régions aide-t-elle un agent sur de longues tâches multi-fichiers ?* — nécessite in fine des déroulements de LLM (§10). Mais sa *condition nécessaire* est testable dès maintenant, sans LLM : **un agent ne peut résoudre une tâche dont le contexte causal requis est absent de sa fenêtre**. Nous mesurons, sous un oracle explicite, si chaque stratégie de sélection *place le contexte causal requis dans la fenêtre*.

Dispositif.

Nous générons des dépôts synthétiques modulaires : S sous-systèmes indépendants, chacun un ensemble de fichiers reliés par une chaîne causale inter-fichiers plus des symboles *leurres* à haut score ; il n'y a pas d'arêtes entre sous-systèmes, donc chaque sous-système est une région causale bornée. La structure causale est *découplée* de la structure lexicale — un voisin de chaîne ne partage aucun jeton d'identifiant avec la cible, seulement une arête — modélisant une dépendance causalement essentielle mais lexicalement dissemblable. Une tâche de *diamètre* d requiert la fenêtre $\pm d$ le long de sa chaîne (jusqu'à $2d + 1$ fichiers) ; le budget contient $\approx$ un sous-système, bien moins que le dépôt. L'**oracle** est $R(t) \subseteq S$.

Surtout, les pipelines de récupération localisent le code à partir d'une *requête* textuelle, tandis qu'une mémoire de type OS *s'ancre* sur le signal de l'espace de travail (le fichier actif, un test en échec). Nous modélisons les deux : chaque tâche porte une requête (un sac de jetons) et une ancre (le nœud du fichier actif), et nous exécutons deux scénarios —

propre (la requête pointe vers la cible) et **bruité** (un leurre piège dans un sous-système sans rapport surclasse lexicalement la cible). Six stratégies se partagent le budget : rag-dense (top- k lexical), rag-hybrid (lexical + score causal), graphrag-1hop (meilleur résultat + un saut), graphrag-bfs (expansion non bornée depuis le meilleur résultat), ccos-from-query (région CCOS du meilleur résultat *lexical* — une ablation), et ccos-region (région CCOS de l'*ancree*). Amorcées et déterministes ; reproduire avec ccos experiment.

Simulation d'hypothèse (800 tâches, graine 42, budget $\approx$ un sous-système). Succès = l'ensemble causal requis $R(t)$ est dans la fenêtre. Sous une requête propre, toute méthode sensible à la structure fait jeu égal ; sous une requête trompeuse, seule la région ancree sur l'espace de travail survit.

	succès au diamètre d				global	
2-5(lr)6-7 Stratégie	$d = 1$	$d = 2$	$d = 3$	$d = 4$	succ.	couv.
<i>Requête propre (pointe vers la cible)</i>						
rag-dense	0.00	0.00	0.00	0.00	0.00	0.19
rag-hybrid	1.00	0.00	0.00	0.00	0.23	0.65
graphrag-1hop	1.00	0.00	0.00	0.00	0.23	0.58
graphrag-bfs	1.00	1.00	1.00	1.00	1.00	1.00
ccos-from-query	1.00	1.00	1.00	1.00	1.00	1.00
ccos-region	1.00	1.00	1.00	1.00	1.00	1.00
<i>Requête bruitée (un leurre surclasse lexicalement la cible)</i>						
rag-dense	0.00	0.00	0.00	0.00	0.00	0.19
rag-hybrid	1.00	0.00	0.00	0.00	0.23	0.65
graphrag-1hop	0.00	0.00	0.00	0.00	0.00	0.19
graphrag-bfs	0.00	0.00	0.00	0.00	0.00	0.00
ccos-from-query	0.00	0.00	0.00	0.00	0.00	0.00
ccos-region	1.00	1.00	1.00	1.00	1.00	1.00

Constats (et leurs limites honnêtes).

(1) La récupération lexicale (rag-dense) *échoue* sur les tâches causales inter-fichiers (0% ; couverture 0, 19) : la similarité ne peut faire émerger un contexte causalement essentiel mais lexicalement dissemblable — la prémisse de l'hypothèse. (Les victoires à $d = 1$ de rag-hybrid viennent du *score causal* qu'il emprunte, non de la similarité lexicale, raison pour laquelle elles persistent sous le bruit.) (2) La valeur de la sélection sensible à la structure *croît avec le diamètre* : l'expansion à un saut ne résout que $d = 1$, tandis que la pagination causale complète (graphrag-bfs, ccos-region) résout tous les d — la direction de H2. (3) **Le jeu égal en cas propre.** Sous une requête propre, graphrag-bfs, ccos-from-query et ccos-region atteignent toutes 1, 00 : le levier est la *structure* causale, **pas CCOS en soi.** (4) **La séparation en cas bruité.** Sous une requête *trompeuse*, toute méthode qui localise le code lexicalement s'effondre à 0% — y compris le robuste graphrag-bfs et l'ablation ccos-from-query (CCOS amorcé sur la requête). Seule ccos-region, qui s'ancree sur le signal de l'espace de travail plutôt que sur la requête, survit à 1, 00. L'ablation isole le différenciateur : c'est la *source de l'ancree* (un signal structurel d'espace de travail contre une requête lexicale), non la machinerie

des régions. C’est le régime réaliste — une description de tâche nomme rarement la cause distante exacte, et une mémoire de type OS qui suit le working set actif est robuste là où la récupération au moment de la requête est trompée. (5) **Hypothèses, déclarées.** Cela crédite CCOS d’une ancre fiable (le fichier actif / le test en échec), qu’une mémoire au niveau OS possède mais qu’un pur pipeline de récupération n’a pas ; et cela suppose des dépôts *modulaires* aux régions séparables (densité 0,955 sur du vrai code, §8) — une région géante monolithique dépassant le budget effondre l’avantage (observé, rapporté). (6) Tout au long, c’est une *simulation sous un oracle déclaré* : elle teste la condition *nécessaire* (récupération), non la condition *suffisante* (génération) de la §10.

Évaluation LLM réelle : une première mesure embarquée

La simulation (§9) a établi la condition nécessaire. La condition suffisante — qu’un agent LLM *résolve* ensuite plus de tâches avec une mémoire par régions qu’avec une récupération par fragments — requiert des déroulements de modèle. Nous **implémentons le harnais** (ccos eval) et rapportons un **premier essai LLM réel** contre un modèle local.

Harnais implémenté.

Chaque tâche est un minuscule projet multi-fichiers encodant une *chaîne causale arithmétique* : une constante de base dans un fichier est transformée à travers une chaîne de fonctions d’une ligne réparties dans des fichiers distincts, et la question « quel entier la dernière fonction renvoie-t-elle ? » n’est répondable *qu’en* lisant toute la chaîne (la cause distante comprise). La notation est donc une correspondance exacte sur un entier — objective et automatisable, sans exécution de code. Les six mêmes stratégies de la §9 assemblent la fenêtre de contexte sous un budget de jetons (avec la division de requête propre/bruitée), la fenêtre est envoyée à un modèle (tout point d’accès compatible OpenAI, Anthropic-Messages ou Ollama), et nous enregistrons trois métriques par diamètre : le **succès de tâche** (entier correct), la **couverture oracle** (chaîne requise $\subseteq$ fenêtre — indépendante du modèle), et l’**hallucination de symbole** (la réponse cite une fonction absente du projet).

Dispositif.

Nous exécutons ccos eval en embarqué sur une NVIDIA Jetson AGX Thor contre qwen2.5:7b-instruct servi par Ollama (température 0), avec 20 tâches, graine 7, un budget de 600 jetons, des diamètres de 1 à 4, dans les deux régimes propre et bruité. Le Tableau 3 rapporte l’essai.

Premier essai LLM réel : qwen2.5:7b-instruct via Ollama, en embarqué (NVIDIA Jetson AGX Thor), 20 tâches, graine 7, budget 600 jetons. « couv. » est la couverture oracle indépendante du modèle ; « jet. » la moyenne des jetons d’entrée. Le succès est une réponse entière en correspondance exacte.

succès au diamètre d					
2-5 Stratégie	$d = 1$	$d = 2$	$d = 3$	$d = 4$	couv. jet.

succès au diamètre d						
<i>Requête propre (nomme la fonction cible)</i>						
rag-dense	0.12	0.00	0.00	0.00	0.00	519
rag-hybrid	0.00	0.00	0.00	0.00	0.00	521
graphrag-1hop	1.00	0.00	0.00	0.00	0.40	270
graphrag-bfs	1.00	0.00	0.17	0.00	1.00	402
ccos-from-query	1.00	0.00	0.17	0.00	1.00	402
ccos-region	1.00	0.00	0.17	0.00	1.00	402
<i>Requête bruitée (un leurre surclasse lexicalement la cible)</i>						
rag-dense	0.00	0.00	0.00	0.00	0.00	531
rag-hybrid	0.00	0.00	0.00	0.00	0.00	521
graphrag-1hop	0.00	0.00	0.00	0.00	0.00	165
graphrag-bfs	0.00	0.00	0.00	0.00	0.00	165
ccos-from-query	0.00	0.00	0.00	0.00	0.00	165
ccos-region	1.00	0.00	0.17	0.00	1.00	402

Constats.

(1) La couverture se transfère de la simulation au texte réel. La colonne de couverture indépendante du modèle reproduit le Tableau 2 sur du contenu de fichier réel : le RAG lexical couvre 0, les méthodes sensibles à la structure couvrent 1,00 sous une requête propre, et sous le bruit *seul* ccos-region maintient 1,00 — la condition nécessaire, confirmée hors du simulateur. **(2) Le succès est borné par le modèle, non par le contexte.** Là où la couverture est 1,00, le modèle de 7B ne résout encore que $d = 1$ (1,00) et une partie de $d = 3$ (0,17), manquant $d = 2, 4$: avec toute la chaîne dans la fenêtre, l'échec résiduel est le *raisonnement arithmétique* — un plafond de capacité du modèle, non un échec de sélection. Surtout, aucune méthode ne réussit jamais *sans* couverture (succès $\subseteq$ couverture), ce qui est l'affirmation que le harnais est bâti pour tester. **(3) La séparation en cas bruité tient sur un vrai LLM.** Sous une requête trompeuse, toute méthode pilotée par la requête s'effondre à 0% de succès — y compris graphrag-bfs et l'ablation ccos-from-query — tandis que ccos-region, ancrée sur le signal de l'espace de travail, est la *seule* stratégie à succès non nul ($d = 1$ à 1,00). Les méthodes pilotées par la requête paraissent même *moins coûteuses* sous le bruit (165 contre 402 jetons) précisément parce qu'elles récupèrent avec assurance le mauvais fichier, plus petit ; ccos-region dépense son budget sur la chaîne causalement correcte. **(4) Limites honnêtes.** 20 tâches (≈ 5 par diamètre) est un petit échantillon, et un modèle de 7B met au plancher la condition suffisante. Un modèle de pointe (deepseek-v4-pro, en cours) ou un modèle local de 70B devrait élever le succès partout où la couverture est 1,00, en *affinant* — non en changeant — la séparation, puisque la couverture fixe déjà le plafond.

Vers des benchmarks externes.

La suite à chaîne arithmétique isole la question de la sélection ; les tests externes décisifs sont la résolution d'issues SWE-bench (un correctif passe les tests cachés) et une suite contrôlée de *bogues multi-fichiers* où le site de la faute, sa cause et son rayon d'impact se trouvent dans des fichiers distincts. Les références partagent le LLM de base et le budget de jetons : RAG classique (top- k cosinus sur fragments), GraphRAG (résumés de communautés sur un graphe de code), MemGPT (mémoire paginée de type OS), un

agent LangGraph avec un magasin vectoriel, et les régions CCOS. Métriques : taux de résolution, jetons d'entrée jusqu'au succès, et un taux d'hallucination d'ancrage de citation vérifié contre le graphe vérité-terrain. Nous avons commencé ceci sur l'historique réel : `scripts/causal_validation` extrait des commits de correction, récupère l'arbre d'avant-correctif, injecte la faute, et score $R_{\text{cov}} = |F_{\text{target}} \cap \text{WorkingSet}_K| / |F_{\text{target}}|$ — la fraction des fichiers qu'un correctif a touchés que le working set borné récupère. Sur l'historique de ce dépôt, le premier essai a exposé une limitation et sa correction, toutes deux mesurées : avec une propagation d'échec uniquement en aval, R_{cov} était plat à 0,33 (seul le fichier de départ récupéré, car les fichiers co-modifiés sont des importateurs *en amont* atteints seulement par des concentrateurs de dépendances). Résoudre les importations intra-crate en arêtes fichier → fichier et propager l'échec *de façon bidirectionnelle* corrige cela. Sur trois crates matures — fd, bat et hyperfine, 70 commits de correction extraits — l'effet est cohérent : à budget suffisant ($K \geq 50$) les deux changements élèvent R_{cov} à 0,85–1,0 (depuis 0,50–0,84 en aval seul), tout en se *diluant* à 0,19–0,28 à un budget serré $K = 20$. **Mais face à la référence évidente, ce n'est pas une victoire.** En exécutant un RAG lexical classique (cosinus TF-IDF sur le texte des fichiers, interrogé par le fichier de la faute) au *même budget de fichiers*, R_{cov} pour CCOS/RAG est de 0,92/0,94, 1,00/0,98, 0,87/0,92 à $K = 50$ et fait jeu égal à $K = 100$, tandis qu'à $K = 20$ le RAG est nettement devant (0,20/0,56 sur bat, 0,20/0,73 sur hyperfine). La sélection causale n'a donc *aucun avantage net de couverture* sur la similarité lexicale ici, et est pire à budget serré : sur des bogues réels, les fichiers d'un correctif se ressemblent lexicalement, si bien que le TF-IDF les récupère aussi — la prémisse de la §9 d'une cause lexicalement dissemblable de son symptôme ne se reproduit pas sur ces dépôts. La lecture honnête est que la condition *nécessaire* tient pour la grande majorité mais n'est *pas* spécifique à CCOS ; un avantage réel devrait venir des régimes que ce dispositif ne teste pas — une requête dégradée/absente (§9), un symptôme (test en échec) lexicalement loin de sa cause (nécessitant le vrai amorçage piloté par le test, non l'heuristique du plus haut degré), ou la condition *suffisante* (Phase 4). Les espaces de travail multi-crates, désormais liables, restent aussi à mesurer à grande échelle.

Condition suffisante (Phase 4) : la résolution fait jeu égal, l'efficacité non.

Nous exécutons la moitié génération sur de vraies corrections mono-fichier de fd : un agent reçoit un contexte à budget égal bâti de deux façons — la région causale de CCOS contre un RAG lexical top- k — on lui demande de réécrire le fichier bogué, et il est noté selon que `cargo test` passe, avec une nouvelle tentative *compilateur dans la boucle* (à l'échec, une *faute de page de contexte* analyse l'erreur avec la couche de trace, injecte de la pression sur les fichiers fautifs, et réamorce avec une fenêtre rafraîchie). Avec un faible modèle de 7B et un seul essai, les deux résolvent 0/15 ; avec qwen3-coder-30B et trois tentatives, la boucle élève les deux à 2/10 (20%) — c'est le retour par faute de page, non le récupérateur, qui débloque la résolution, et CCOS ne montre **aucun avantage de résolution** sur le RAG, cohérent avec le résultat de récupération. L'*efficacité* les sépare : à résolution égale, la région auto-bornée de CCOS fait en moyenne 776 jetons de contexte contre les 5366 qui remplissent le budget pour le récupérateur lexical — une réduction de **6,9** ×. La même chose tient à l'échelle sans modèle : sur 51 scénarios de correction mono-fichier issus de fd, bat et hyperfine, CCOS assemble 700 à 1600 jetons de contexte contre ≈ 6000 pour le RAG, une réduction de **4,1** à **9,1** ×. C'est le seul axe sur lequel CCOS domine : non pas *ce qu'il* récupère (une référence l'égale) mais *combien peu* il lui faut, parce que la région causale s'arrête au working set au lieu de remplir un top- k jusqu'au budget. Nous énonçons les réserves : l'échantillon de *résolution* est minuscule ($n = 10$, deux passes), et la référence remplit le budget par construction (un RAG à k soigneusement réglé serait aussi plus clairsemé). L'affirmation

défendable est plus étroite mais réelle : CCOS s'*auto-calibre* — il se borne à la région causale sans k ni budget à régler — donc il ne gaspille jamais la fenêtre, ce qui est précisément le but de la pagination à la demande.

Hypothèses.

H1 (efficacité) : CCOS atteint un succès de tâche égal avec moins de jetons d'entrée que RAG/GraphRAG, parce qu'une région couvre le voisinage causal avec un meilleur rappel (Tableau 1). **H2** (succès à long horizon) : sur les tâches multi-fichiers, le taux de résolution de CCOS dépasse le RAG par fragments d'une marge qui croît avec le diamètre causal de la tâche. **H3** (ancrage) : CCOS abaisse le taux d'hallucination de symbole, parce que la fenêtre admise est un sous-graphe connexe de nœuds *réels*.

Menaces à la validité.

La qualité d'une région est bornée par la qualité des arêtes (§8) ; les résultats confondraient la *politique* de région avec le *graphe* sous-jacent. Les ablations doivent donc fixer le graphe et ne varier que la stratégie de sélection, et rapporter le diamètre causal par tâche pour que les gains soient attribués à la cohérence, non au fait de récupérer plus de texte. L'ablation *ccos - from - query* du Tableau 3 est exactement ce contrôle — même graphe et même budget, seule l'ancre échangée — et elle s'effondre avec les références lexicales sous le bruit. Un résultat positif sur H1–H3 constituerait la contribution de recherche ; un résultat nul validerait quand même l'infrastructure déterministe et auditable.

Limites

Nous sommes délibérément explicites sur ce qui n'est *pas* montré. **(1) Aucun avantage de couverture sur le RAG pour des bogues réels.** La métrique phare sur données réelles (R_{cov} , §10) fait jeu égal avec un simple récupérateur lexical TF-IDF à budget suffisant et lui est inférieure à budget serré ; sur de vrais commits de correction, les fichiers qu'un correctif touche se ressemblent lexicalement, donc la prémisse de la simulation d'une cause lexicalement dissemblable ne tient pas. Les chiffres absolus forts sont une condition *nécessaire* qu'une référence satisfait aussi, non une victoire de CCOS. (Une mesure complémentaire sur le code source du moteur lui-même isole une relation différente, définie structurellement : un récupérateur lexical TF-IDF ne retrouve que 49% (rappel@10) des dépendances d'*import* résolues par l'AST — des paires qui ne partagent souvent aucun vocabulaire, cosinus de dépendance 0,53 contre 0,43 aléatoire — de sorte que l'angle mort du RAG est réel, mais il se situe hors de l'axe de *cohésion* d'un correctif que mesure la métrique phare, où les fichiers d'un correctif partagent bien leur vocabulaire. La couche structurelle retrouve ces arêtes par construction ; l'AST désormais par défaut est ce qui rend cette récupération complète.) **(2) Nécessaire ≠ suffisant.** Nous ne mesurons jamais si une région *aide un agent à corriger* un bogue (Phase 4 : un correctif qui passe les tests cachés) — seulement si les fichiers pertinents sont récupérables. **(3) Heuristique d'amorçage.** Le harnais injecte la faute dans le fichier modifié de plus haut degré sortant, non dans le test réellement en échec, donc il n'exerce pas le régime où un symptôme est lexicalement loin de sa cause — le régime le plus favorable à la sélection causale. **(4) Échelle et statistiques.** Trois dépôts mono-crate, $n \approx 20$ chacun ; les différences sont rapportées avec leur écart-type mais non validées de façon croisée. **(5) Moteur.** Le parseur utilise désormais *par défaut* un vrai AST syn —

mesuré 36, 5% plus précis que l’ancienne heuristique ligne sur le code source du moteur lui-même (deux tiers contre la totalité du rappel d’imports), l’heuristique n’étant conservée que comme repli pour les entrées non-Rust — et l’ingestion a été durcie pour se reconstruire bit à bit à l’identique entre processus (résolution d’imports triée ; centralité invariante à l’ordre). Les régions restent à granularité fichier et ne fusionnent que sur des arêtes inter-fichiers explicites ; les poids de score sont réglés à la main, bien que des raffinements optionnels (désactivés par défaut) de centralité par vecteur propre et d’états de cycle de vie des nœuds (Stable/Working/Orphan) soient désormais disponibles. Rien de tout cela n’affecte les propriétés *prouvées* (partition, déterminisme, rejou) ni la localité mesurée — mais le dossier empirique d’un bénéfice pour l’agent en aval est, à ce stade, *non prouvé*.

Conclusion

Nous étions partis pour montrer qu’organiser la mémoire d’un agent par *régions causales* récupère le contexte à long horizon mieux que la récupération par fragments, avons donné à la construction une définition précise et déterministe, et prouvé qu’elle se reconstruit bit pour bit au rejou. Nous avons ensuite testé l’affirmation de récupération contre la référence évidente sur données réelles — et elle n’a pas tenu : sur 70 commits réels de correction de bogues, un simple récupérateur lexical TF-IDF fait jeu égal avec la sélection causale et la bat à budget serré, et un pivot par trace de crash perd contre un RAG-sur-le-message-d’erreur, parce que sur du vrai code les fichiers d’un correctif et ses messages d’erreur partagent leur vocabulaire. Nous rapportons le résultat négatif plutôt que de l’enterrer. Ce qui survit n’est pas un meilleur récupérateur mais un type d’objet différent : une mémoire de travail *déterministe, rejouable, auditable* dans laquelle l’état exact du contexte d’un agent peut être rembobiné et rejoué sous d’autres paramètres — un débogage par voyage dans le temps qu’une pile de récupération probabiliste ne peut offrir. Que cette audibilité, ou un avantage de contexte structuré au moment de la *génération* (Phase 4), produise un gain mesurable en aval reste ouvert. Nous publions le moteur, le harnais de validation honnête et cet article afin que la distinction entre ce qui est prouvé, ce qui est mesuré et ce qui est réfuté puisse être vérifiée plutôt qu’affirmée.

99 P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020. arXiv:2005.11401. A. Asai et al. Self-RAG: Learning to Retrieve, Generate and Critique through Self-Reflection. *ICLR*, 2024. arXiv:2310.11511. D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. 2024. arXiv:2404.16130. F. Yamaguchi et al. Modeling and Discovering Vulnerabilities with Code Property Graphs. *IEEE S&P*, 2014. C. Packer et al. MemGPT: Towards LLMs as Operating Systems. 2023. arXiv:2310.08560. J. S. Park et al. Generative Agents: Interactive Simulacra of Human Behavior. *UIST*, 2023. arXiv:2304.03442. LangChain. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. Framework logiciel, 2023–2024. <https://github.com/langchain-ai/langgraph>. P. J. Denning. The Working Set Model for Program Behavior. *Communications of the ACM*, 11(5), 1968. N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. *TACL*, 2024. arXiv:2307.03172. W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*, 2023. arXiv:2309.06180. S. Haber and W. S. Stornetta. How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2), 1991. C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024. arXiv:2310.06770.

1. Causal Context Operating System (CCOS), un prototype de recherche ouvert. Le code source, les scripts de reproduction et cet article : <https://github.com/CHECKUPAUTO/CCOS>.↵